THE AI AGENT FACTORY · by Panaversity

General Agents on the Web

Your first general agent: **Claude Cowork on the web**, nothing installed. The whole course on six pages: concepts, hands-on tasks, safety rules and habits.

SESSION 1 · PAGE 2

Chat versus delegated work

- Chat box vs agent surface
- Remote session: the tab is a window
- Scheduled tasks: heartbeats

Hands-on 1 to 3

SESSION 2 · PAGES 3-4

Persistence, files, reach, gates

- Memory stack and 3 file tiers
- Connector, browser, computer use
- Prompt injection rules
- Manual / Auto / Skip, blast radius

Hands-on 4 to 7

SESSION 3 · PAGE 5

Working unwatched

- Delegation loop
- The 8-part brief
- Reviewing the plan
- Six answers before scheduling

Hands-on 8 to 9

SESSION 4 · PAGE 6

Choosing, and the open path

- Web, desktop or terminal routing
- Managed vs open path
- Portability and capstones
- The ten habits

Hands-on 10 to 11

Chat waits. Delegated work continues.

Chat versus delegated work

Same address (claude.ai), two different things. One question tells them apart.

"If I stop typing, does the work stop?"

Chat box: yes. Agent surface: no. This one question separates the two at any address, from any company.

The chat box

Waits for you every turn. It cannot start on a schedule, react to an event, or continue after you close the tab. That is what a conversation is: where you **think, draft and design**.

The agent surface

You give an **assignment, not a message**. It makes a plan, uses tools, works through steps and produces a deliverable. It pauses only where a decision needs you, and **keeps working when you stop watching**.

The remote session: the tab is a window

A Cowork task started on the web runs on **Anthropic's servers, not in your browser**. One session = one ID in your account. It plans, uses connectors and web tools, writes files, pauses when it needs you.

- **Tab open:** you watch it work
- **Laptop closed:** it keeps working
- **Phone:** same session, read the result or approve a step

One exception: with Claude Desktop, a cloud task can borrow a local folder or browser. The borrowed tool stops when the computer is off; the session does not.

Scheduled tasks: heartbeats

A schedule is the heartbeat: something other than your next chat turn starts the work.

- **Once:** run later, one time (a reminder for tomorrow)
- **On a schedule:** fires on a clock, laptop closed (every Monday 8am)
- **Monitor:** checks repeatedly, surfaces a result only when a condition becomes true

Key rule: a schedule is device-independent only if every required tool is reachable without your device. A Monday brief that needs a file on your sleeping laptop has rebuilt the laptop as a hidden dependency.

Hands-on 1: Chat versus Cowork

Give the same job to chat and to Cowork. Watch which one keeps working after you stop typing; a finished document should exist that chat never produced.

Hands-on 2: The tab is a window

Start a Cowork cloud task (Drive folder or web page source). Close the tab, then find the same session still running on your phone.

Hands-on 3: A run without you

Schedule "run this ten minutes from now and notify me". Close the laptop and watch the result arrive on your phone.

If I stop typing, does the work stop? Chat: yes. Cowork: no.

Persistence and files

Persistent context is a stack, and every deliverable lands in one of three tiers.

The account spine: five persistent layers

Layer	The question it answers	Lifetime
Session history	What happened in this particular task?	One task
Project	What belongs to this continuing body of work?	Weeks, months or longer
Semantic memory	What facts and preferences carry across cloud sessions?	Until edited, reset or disabled
Standing instructions	How should the assistant generally behave here?	Until you or an admin change them
User-owned context file	What do I want portable, inspectable, vendor-neutral?	As long as you maintain it

Rules to remember

- An old session is not memory. A Project is not memory. They all persist, but for **different reasons**.
- Put information in the **narrowest layer that reliably applies**: a one-client rule belongs in that client's Project, not in a global instruction.
- Local Cowork sessions do not use cloud memory.

The three file tiers: where does the deliverable land?

Tier 1 · Task filesystem (custody: theirs)

Temporary working space while the session runs: drafts, scratch math, half-built files. **Wiped when the task ends.** Never the home of anything you need tomorrow.

Tier 2 · Platform storage (custody: theirs)

Saved to your account; survives the task, the tab and the week. The **default landing place**, in the vendor's custody, format and login.

Tier 3 · The exit (custody: yours)

The file leaves the platform into a system you control: connector save to Drive, email, download, local write, repo commit. **The destination matters, not the door.** Nothing reaches tier 3 unless the brief asks for it.

Hands-on 4: Memory and instructions

Open memory settings and edit or delete one stale topic. Create a Project with one local instruction (e.g. "updates are one page"), run a task inside it, and name the layer each thing lives in.

Hands-on 5: The three-tier audit

Brief a one-page summary of a Drive document, ending with "List every file you created and where each one landed". Then send the finished file to tier 3 and open it in Drive, not Cowork.

Finished work exits the platform. Everything else may stay.

Reach, safety and gates

How an agent reaches the world, what can go wrong, and how much it may do without you.

Reach path	Best for	Main advantage	Main risk or limit
Connector	Gmail, Drive, Slack, CRMs, databases	Structured permissions you can read before granting	Scope can be too broad; results can carry prompt injection
Browser	Portals, forms, dashboards, sites with no connector	Reaches pages no connector covers	Authenticated browsing raises action risk; Claude in Chrome inherits your logins
Computer use	Apps with no connector or browser UI	Reaches the GUI directly	Highest risk; needs Claude Desktop

Two browser paths, two jobs

Claude in Chrome: "help with this page I already have open", in your own Chrome with your logins. **Built-in Cowork browser:** "go handle that site for me", in Claude's separate browser (needs Claude Desktop, online). **Connector before browser. Browser before full computer use.**

Prompt injection: read + act together

Your brief is authority. External content is evidence. Instructions found inside emails or web pages are untrusted input, never a new boss. Six rules:

1. Start with trusted sites, low-stakes accounts
2. Never reuse banking, medical or identity logins
3. Manual approval for unfamiliar sites, new plugins/connectors, and anything that sends, spends, deletes, publishes or changes records
4. Give only the reach a workflow needs
5. Connectors as entry and exit: name the destination
6. Install only trusted plugins

Read is not send, draft is not publish, view is not edit.

Three Cowork modes

Manual: Claude pauses for actions needing approval. For new workflows, unfamiliar sites, sensitive or hard-to-reverse actions.
Auto: keeps moving, screens each action for safety, blocks or escalates unsafe ones; uses more allowance. For proven, bounded workflows.
Skip: no pauses, no action check. Rarely for beginners; never in the labs. "Automatically approve" is a product default, not a recommendation.

Match the gate to blast radius

Low (read and summarise): automatic is often fine.
Medium (draft, do not send): output is reviewable first.
Higher (write to reversible records): stronger gate until proven.
High (send, publish, purchase, delete): Manual or explicit human control.
Choose the gate **before the run**, not when the prompt appears.

Hands-on 6: Connectors and scopes

Open the Drive (optionally Gmail) connector and read exactly what it may do. Ask a task to do something outside the scope and watch it refuse. Widen nothing.

Hands-on 7: The gate lab

Run a bounded workflow in Manual (count interruptions), then Auto (what cleared vs escalated). Then ring the gate: add a step needing you, walk away, approve from your phone.

Match the gate to blast radius. High-consequence work earns tighter gates.

Working unwatched

The product can work without you, so the quality of the handoff matters more, not less.

Workflow discovery. When the workflow is unclear, start with your frustration: ask Cowork to interview you (goal, sources that matter, what must never change, what a good finished brief looks like), then **propose the workflow before doing it**.

Step 1 · Brief

Outcome, audience, sources, constraints, permissions, file destinations, definition of done.

Step 2 · Plan (the leverage)

Ask to see the approach before consequential work. Put it in the brief: Manual mode does not create a plan for you.

Step 3 · Approve or redirect

Fix the plan while changes are cheap. Set the permission mode to match the stakes.

Step 4 · Review

Inspect the deliverable, the actions that mattered, and where every file landed.

The 8-part brief for work you will not watch

1 Outcome: what finished thing exists **2 Audience and use:** who, for what decision **3 Sources:** files, Projects, connectors, sites allowed **4 Boundaries:** what it must not read, change, send, purchase, delete, publish **5 Method constraints:** format, calculations, citations, date window, rules **6 Escalation:** what uncertainty stops the run **7 Destination:** name the tier-3 system **8 Done:** observable conditions.
The line beginners omit: "If the necessary evidence is missing, say what is missing and stop rather than inventing it."

Review the plan, not every click

Scope: only the work and data I named?
Order: does it verify before it acts?
Reach: any connector, send, publish, write or delete I did not intend?
Assumptions: is it silently deciding audience, format, rule or a missing fact?
Redirect with **one sentence**.

Six answers before you schedule

1 What starts the run (clock, event, watch)? 2 What does it touch? 3 Can it reach everything with your devices off? 4 How will you know it succeeded? 5 What may it do without approval? 6 What happens on the empty case ("no new items" is a valid result)? **Walk the workflow by hand twice first.**

Ran is not the same as worked

Ran = runtime evidence: it fired, finished, an output exists. **Worked** = outcome evidence: the success condition was checked and the result reached its destination.

Hands-on 8: Frustration to approved plan

Interview, 8-part brief, "show me your plan first", run in Manual, redirect once, review.

Hands-on 9: The first cloud schedule

Answer the six questions, walk it by hand twice, then attach the heartbeat to real work.

Fix the plan, not the cleanup. Scheduling photocopies a workflow into the future.

Choosing, and the open path

Route by what the work touches, own what travels, and keep ten habits.

Work pattern	Best starting surface	Why
One-off thinking, drafting, workflow design	Chat	Nothing multi-step to delegate
Cloud files and connectors, device may be off	Cloud-only Cowork	No local dependency; strongest unattended continuity
Cloud work needing one approved local folder or browser	Cloud + desktop bridge	Cloud continuity, local tool borrowed while online
Local files and apps, local custody matters	Local desktop agent	The machine is the execution surface
A repository, terminal, tests or CI	Coding agent or terminal	The repo and toolchain are the system of record
A website with no connector	Browser path	Built-in browser to delegate; Claude in Chrome for an open page
Regulated or contractually restricted data	Stop and verify	Availability is not permission; get a written compliance answer first

Three routing questions

1 What does the work touch? **2** Must it run while my device is off? **3** Who may hold and process the data? Do not choose a tool because it is newer. A bridge is not a custody loophole: a local file read through Claude Desktop is processed in the cloud.

Managed surface vs open path

Managed: a furnished office (sessions, sync, phone gate, scheduler) in their custody and price. **Open:** an empty room you furnish: OpenWork with your own worker, or OpenCode on cron / GitHub Actions. You gain custody and model choice, but build the spine yourself.

Capstone projects

Project 5 (optional): same task in Cowork and ChatGPT Work; compare what starts it, reaches, remembers, needs you, runs.
Project 6: portability drill: rebuild one workflow from tier-3 files only and list what you could not.

Hands-on 10: Route your own week

Write 3 real tasks (cloud docs, local-only files, a site with no connector), route each with the three questions, mark regulated data "stop and verify".

Hands-on 11: The portability test

Write brief, instructions, sources and definition of done into one plain-text file; save to Drive; read it with Cowork closed; list what would not travel.

The habits in one place

Ten habits: 1 Start with the frustration and let the agent interview you 2 Brief the workflow in full 3 Catch problems at the plan 4 Connector over browser, browser over computer use 5 Match the gate to blast radius 6 Send finished work to tier 3 7 Schedule only workflows you have walked 8 Ask where the loop runs and each tool executes 9 Read new products by parts: what starts it, reaches, remembers, needs you, where it runs 10 Take long-term runtime and custody decisions to "Leaving the Laptop"

Chat is where you converse. The agent surface is where you delegate.